esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

RAPPORT DE STAGE

STAGE INGÉNIEUR

DUREE DE STAGE :

DU 16 JUIN 2025

JUSQU'AU 15 AOUT 2025

Créé par : Ghassen Ben Aissa

Encadré par : Mr. Houssem Jmal

EUR-ACE® Délivrée par la Commission des Titres d'Ingénieur
CONFÉRENCE DES GRANDES ÉCOLES
cdio
AUF Agence Universitaire de la Francophonie
IFGU

Remercîment

Je tiens à remercier mon encadreur, Mr Houssem Jmal.
Un grand merci pour son accueil chaleureux, ainsi que pour sa patience et ses conseils précieux. Grace à sa confiance, j'ai pu m'accomplir totalement dans mes missions.

Table des matières

Pages Liminaires	**Page de garde**	**1**
	Remerciements	**2**
	Table des matières	**3**
	Lisre des Figures	**4**
	Liste des Tableaux	**5**
Corps du Rapport	**Introduction Générale**	**6**
Chapitre 1	**Présentation de l'Entreprise et du Contexte**	**7**
	1.1 Présentation générale et secteur d'activité	**7**
	1.2 Organisation et positionnement du service d'accueil	**8**
Chapitre 2	**Définition de la Mission et Problématique**	**9**
	2.1. Description détaillée de la mission de stage	**9**
	2.2. Choix Technologiques et Outillage	**10**
	2.3. Problématique retenue pour le rapport	**11**
Chapitre 3	**Développement des Travaux et Réalisations Techniques**	**12**
	3.1. Méthodologie et outils utilisés	**12**
	3.1.1. Planification et Découpage du Travail	**12**
	3.1.2. Gestion du Backlog	**14**
	3.1.3. Organisation des Sprints et Rôles d'Équipe	**14**
	3.2. Conception et Implémentation du Socle Technique Monolithique	**16**
	3.2.1. Mise en Place du Backend avec Spring Boot (Java 21)	**16**
	3.2.2. Sécurité et Gestion des Accès	**17**
	3.2.3. Développement Frontend Modulaire avec Angular 16	**17**
	3.3. Développement des Fonctionnalités Métier	**20**
	3.3.1. Le Système de Communication (WebSockets)	**20**
	3.3.2. Architecture d'Intégration de l'Intelligence Artificielle	**21**
	3.4. Bilan des Résultats Obtenus	**23**
Conclusion	**Conclusion**	**26**

Liste des Figures

1 Logo Talan **7**

2 Logo Angular **10**

3 Logo Spring boot **10**

4 Logo PostgreSQL **10**

5 Logo IA **10**

6 Diagramme de classe **13**

7 Interface de connexion avec gestion des rôles utilisateurs (Admin, Expert, Client) **18**

8 Interface de navigation dynamique selon le rôle de l'utilisateur **18**

9 Détails du profil expert avec biographie, compétences et domaine d'expertise **19**

10 Système d'étoiles pour l'évaluation d'un expert après une mission **19**

11 Dashboard expert affichant les statistiques et les demandes reçues **19**

12 Espace de discussion entre client et expert (messagerie en temps réel) **20**

13 Notification reçue lors d'une nouvelle demande de communication ou d'un message entrant **20**

14 Formulaire de recherche textuelle par le client (rédaction du besoin) **22**

15 Liste des experts recommandés avec score de similarité **22**

16 Interface affichant l'ensemble des domaines disponibles sur la plateforme **23**

17 Présentation des experts inscrits, avec leurs compétences principales **24**

18 Détails du profil expert **24**

19 Liste des demandes (côté expert) **24**

20 Détails du client (partie administrateur) **25**

21 Détails de l'expert (partie administrateur) **25**

22 Liste des domaines (partie administrateur) **25**

Liste des Tableaux

1 User Stories du projet “Khebra” 14

2 Planification des Sprints 15

Introduction Générale

Le cycle de formation d'ingénieur en informatique de l'École Supérieure Privée d'Ingénierie et de Technologies ESPRIT prévoit, pour les étudiants de deuxième année, un stage ingénieur d'une durée de 2 mois en milieu professionnel. Ce stage a pour objectif d'immerger l'élève-ingénieur dans les problématiques concrètes de l'entreprise et de lui permettre d'appliquer les connaissances théoriques acquises.

C'est dans ce cadre que j'ai eu l'opportunité d'effectuer mon stage au sein de Talan Consulting Tunisie, un acteur majeur dans l'accompagnement des entreprises dans leurs projets de transformation et d'innovation. J'ai intégré une équipe projet avec pour mission le développement d'une application de mise en relation, visant à offrir les services d'experts inscrits sur la plateforme à de futurs clients.

Le succès d'une telle plateforme repose non seulement sur sa richesse fonctionnelle, mais surtout sur la robustesse de son socle technique. Les enjeux sont multiples : assurer une expérience utilisateur fluide, gérer une charge potentiellement importante et garantir une maintenance aisée. Ces défis techniques nous amènent à la problématique centrale de ce rapport :

« Face aux exigences de performance, de modularité et d'évolutivité de la plateforme de services, quels choix d'architecture logicielle (frontend/backend, base de données) et quelle méthodologie de développement doivent être adoptés pour garantir la robustesse et la scalabilité de l'application tout en respectant les délais impartis ? »

L'objectif de ce rapport est de présenter l'environnement de travail, d'exposer en détail la démarche adoptée pour le développement de cette plateforme, et d'analyser les solutions techniques retenues en lien avec la problématique posée.

Le présent document est structuré en trois grandes parties : Le Chapitre 1 présentera l'entreprise Talan Consulting Tunisie et le contexte de ma mission. Le Chapitre 2 détaillera la mission et les objectifs fixés, en explicitant la problématique technique. Enfin, le Chapitre 3, le cœur de ce rapport, décrira la méthodologie, les choix d'architecture et les réalisations concrètes obtenues au terme du stage.

Présentation de l'Entreprise et du Contexte

Figure 1 – Logo Talan

1.1. Présentation générale de Talan Consulting Tunisie

Talan est un groupe international de conseil et d'intégration spécialisé dans l'accompagnement des entreprises et des organisations dans leur transformation digitale. Avec une forte présence mondiale, Talan s'est positionné comme un acteur clé dans la transition vers des modèles économiques et opérationnels plus numériques.

Talan Consulting Tunisie, filiale du groupe, joue un rôle essentiel en tant que centre d'expertise et de services. Ses principales activités s'articulent autour de:

- Le Conseil et l'Innovation : Accompagnement stratégique des clients (dans des secteurs variés comme la banque, l'énergie, les télécoms, etc.) pour définir leurs feuilles de route numériques.
- L'Intégration de Systèmes et les Solutions Logicielles : Développement et déploiement de solutions technologiques sur mesure (applicatifs métier, plateformes digitales) et gestion de projets complexes.
- L'Expertise Data et IA : Exploitation des données (Data Science, Intelligence Artificielle) pour optimiser les performances et créer de la valeur.

Mon stage s'inscrit directement dans cette vocation d'intégration de solutions technologiques, au sein d'une équipe dédiée au développement de produits digitaux pour les clients du groupe.

1.2. Organisation et positionnement de l'équipe d'accueil

Mon stage s'est déroulé au sein du pôle Développement et Ingénierie Logicielle, une unité clé de Talan Consulting Tunisie, responsable de la conception et de la livraison des solutions techniques.

Structure de l'équipe projet : L'équipe est structurée de manière Agile, facilitant la collaboration et l'itération rapide. Elle est composée de :

- Un Chef d'Équipe / Chef de Projet : Assurant le pilotage global, la planification des sprints et le lien avec le client.
- L'Équipe de Développement : Qui se divise en deux sous-équipes pour favoriser la spécialisation :
    - Sous-équipe Frontend : Responsable de l'interface utilisateur (UI) et de l'expérience utilisateur (UX).
    - Sous-équipe Backend : Responsable de la logique métier, de l'architecture serveur et de l'interaction avec la base de données.

Positionnement du stagiaire : J'ai été intégré(e) à la fois dans les travaux du Frontend et du Backend, ce qui est très enrichissant pour un architecte logiciel. J'ai bénéficié d'un double encadrement, essentiel à la réussite de ma mission :

- Maître de stage Frontend : M. Houssem JMAL, qui m'a guidé sur les choix d'interface, l'intégration des services et les bonnes pratiques de développement client.
- Maître de stage Backend : Mme Par Ela BEN KHDIMALLAH, qui m'a supervisée sur l'architecture côté serveur, les APIs et la gestion des données.

Cette structure m'a permis d'appréhender le développement d'une application dans sa globalité, de la conception de la base de données à l'affichage final pour l'utilisateur.

Définition de la Mission et Problématique

2.1. Description détaillée de la mission de stage et des objectifs fonctionnels

Le cœur de mon stage a consisté à participer au développement complet d'une plateforme de mise en relation entre des experts et des clients, agissant comme un intermédiaire intelligent pour garantir un Matching optimal. Cette mission a nécessité l'intégration de compétences Full Stack et d'une forte composante en architecture logicielle.

Les objectifs fonctionnels de l'application, réalisés ou initiés durant le stage, se résument aux points suivants :

- Gestion des Utilisateurs (Authentification et Profils) : Mise en place des mécanismes d'inscription, de connexion (Login) et de gestion des profils pour les deux types d'utilisateurs : les Experts (création de leur portfolio de compétences) et les Clients (création de leur compte).
- Recherche Intelligente et Matching : Développement d'un moteur de recherche sophistiqué capable de proposer aux clients les experts les plus pertinents. Cette fonctionnalité repose sur une approche d'intelligence artificielle pour analyser le besoin du client (souvent exprimé en langage naturel) et le mettre en correspondance avec les compétences des experts.
- Système de Communication : Implémentation d'un module permettant la demande de communication (initialisation du contact) et le service de messagerie intégré pour faciliter les échanges directs et sécurisés entre le client et l'expert.
- Évaluation et Qualité de Service : Conception d'un mécanisme d'enquête de satisfaction permettant au client d'évaluer l'expert et la prestation reçue, alimentant ainsi un système de réputation et de tri des profils.

2.2. Choix Technologiques et Outillage

Pour répondre aux enjeux de performance et d'évolutivité de l'application, l'architecture technique a été définie autour de technologies modernes et robustes :

Frontend: **Angular 16**

Figure 2 – Logo Angular

Utilisé pour construire une interface utilisateur dynamique et modulaire, optimisant l'expérience client (UX) et la maintenabilité du code.

Backend: **Spring Boot (Java 21)**

Figure 3 – Logo Spring boot

Choisi pour sa robustesse, sa performance et sa capacité à gérer de grandes charges, typiques d'une plateforme d'entreprise. Il assure la logique métier et l'exposition des APIs REST.

Base de Données: **PostgreSQL**

Figure 4 – Logo PostgreSQL

Base de données relationnelle open source reconnue pour sa fiabilité, sa capacité à gérer la concurrence, et son support avancé des fonctionnalités complexes nécessaires à l'application.

Composante IA: **Modèle KNN (K-Nearest Neighbors) et Sentence Transformer**

Figure 5 – Logo IA

Utilisé pour le tri des experts après la vectorisation des données, permettant une classification rapide et pertinente.

Employé pour la vectorisation sémantique du besoin client et des compétences de l'expert, afin de calculer une similarité de manière intelligente et de fournir un matching plus précis que la simple recherche par mots-clés.

2.3. Problématique retenue pour le rapport

Les choix technologiques et la complexité des fonctionnalités, notamment l'intégration des modèles d'IA dans l'architecture Spring Boot, ont généré des défis techniques spécifiques.

La problématique retenue, qui guidera l'analyse détaillée du chapitre suivant, est la suivante :

« Face aux exigences de performance, de modularité et d'évolutivité de la plateforme de services, quels choix d'architecture logicielle (frontend/backend, base de données) et quelle méthodologie de développement doivent être adoptés pour garantir la robustesse et la scalabilité de l'application tout en respectant les délais impartis ? De manière plus spécifique, comment l'intégration des modèles d'IA (KNN et Sentence Transformer) a-t-elle été conçue et optimisée dans l'architecture Full Stack pour fournir un matching intelligent et performant ? »

Développement des Travaux et Réalisations Techniques

3.1. Méthodologie de Gestion de Projet : Application de Scrum

3.1.1. Planification et Découpage du Travail

La phase initiale du projet a été consacrée à la modélisation pour garantir une architecture logique cohérente avant de démarrer le codage. Le Diagramme de Classes UML (Unified Modeling Language) a constitué la feuille de route fondamentale pour la structure de l'application monolithique, en particulier pour le Backend Spring Boot et la base de données PostgreSQL.

Le diagramme de classes a permis de :

1. Définir les Entités Métier : Il a permis d'identifier les objets centraux du système (Expert, Client, Conversation, Demande, Avis, etc.) et leurs attributs.
2. Modéliser les Relations : Il a formalisé les liens entre ces entités (associations, agrégations, compositions), essentiels pour la mise en place des clés étrangères et des relations JPA (Java Persistence API) dans Spring Boot. Par exemple, une Conversation est associée à un Client et un Expert.
3. Servir de Base aux Entités JPA : Chaque classe du diagramme correspond directement à une entité Java (annotée avec @Entity dans Spring Boot), et chaque relation se traduit par des annotations JPA (@OneToMany, @ManyToOne, etc.). Cela a assuré une correspondance parfaite entre le modèle objet Java et le schéma relationnel de la base de données PostgreSQL.

Afin d'illustrer la structure des données de la plateforme, le schéma ci-dessous présente les principales entités et leurs relations, notamment autour du processus de matching et de communication.

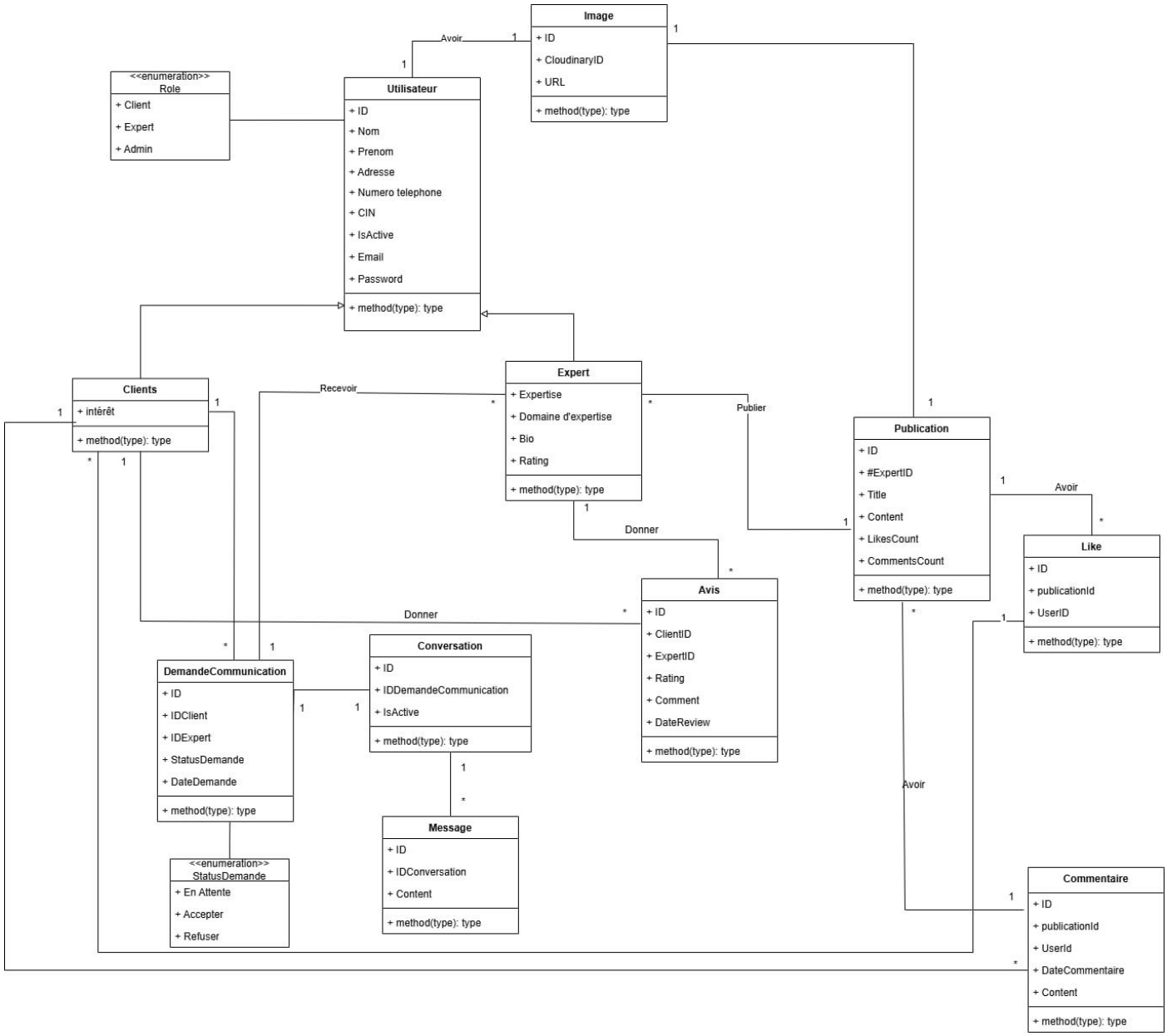

Figure 6 – Diagramme de classe

3.1.2. Gestion du Backlog : Organisation des User Stories

L'ensemble des fonctionnalités a été traduit en User Stories gérées dans l'outil Jira. Ce processus, essentiel à la méthode Scrum, a permis de prioriser les développements et d'assurer l'alignement entre les besoins fonctionnels et la complexité technique.

ID STORY	USER STORY	Priorité	Estimation (en jours)	Sprint
1	En tant qu'utilisateur, je veux pouvoir m'inscrire et me connecter via un formulaire sécurisé.	H	3	SPRINT 1
2	En tant qu'expert, je veux recevoir une notification en temps réel lorsqu'un client m'envoie un message.	H	4	SPRINT 1
3	En tant qu'expert, je veux pouvoir mettre à jour mon profil pour que mes nouvelles compétences soient prises en compte dans la recherche.	M	2	SPRINT 1
4	En tant que client, je veux décrire mon problème en langage naturel pour obtenir une liste d'experts pertinents.	H	7	SPRINT 2

Tableau 1 – User Stories du projet “Khebra”

Le backlog, ainsi détaillé, a servi de base aux réunions de planification de sprint, permettant à l'équipe Front (Angular) et Back (Spring Boot) de s'aligner sur les interfaces d'API à développer dans chaque itération.

3.1.3. Organisation des Sprints et Rôles d'Équipe

La gestion du projet a été structurée autour de quatre Sprints de deux semaines, permettant une progression constante et une livraison incrémentale des fonctionnalités. L'objectif de chaque cycle était de produire une version fonctionnelle des modules les plus prioritaires du backlog.

Sprint	Période	Objectif Principal du Sprint	Réalisations Clés
Sprint 1	Semaines 1 et 2	Socle architectural et sécurité	Modélisation UML, Configuration Spring Boot/Angular, Implémentation de JWT.
Sprint 2	Semaines 3 et 4	Interface Utilisateur et Flux de base	Intégration du *template* Angular, Logique d'authentification Front/Back, Gestion des rôles.
Sprint 3	Semaines 5 et 6	Communication et Qualité de Service	Service WebSocket, Messagerie en temps réel, Système d'avis et d'évaluation.
Sprint 4	Semaines 7 et 8	Intelligence Artificielle et Optimisation	Intégration Sentence Transformer/KNN via FastAPI, Scheduler de mise à jour du modèle, Finalisation UI/UX.

Tableau 2 – Planification des Sprints

Les Rituels de Scrum et la Coordination Front-End / Back-End

L'efficacité du développement, malgré la division de l'équipe entre Front (Angular) et Back (Spring Boot), reposait sur les rituels Scrum, notamment le Daily Scrum.

1. Le Daily Scrum : Cette réunion quotidienne, courte et debout, a joué un rôle de synchronisation essentiel. Elle permettait non seulement de lever les blocages techniques ("Que vas-tu faire aujourd'hui ?") mais surtout d'assurer l'alignement sur les spécifications des API REST.
    - Exemple de Coordination : Si l'équipe Backend développait un endpoint pour la connexion, nous devions nous accorder sur la signature exacte (le format des données attendues en entrée, le format du JSON retourné, et les codes de statut HTTP). Une bonne coordination à ce niveau était vitale pour éviter que l'équipe Frontend n'attende la fin complète du développement du Backend. Ce travail de formalisation des Contrats d'API a été l'une de mes responsabilités majeures en tant que futur Architecte Logiciel.
2. Gestion des API avec Postman (Validation Fonctionnelle) : Bien que l'équipe n'ait pas eu le temps de mettre en place des tests unitaires ou d'intégration automatisés (tests formels), la validation a été assurée par des tests manuels réguliers des endpoints du Backend via Postman.

- Chaque développeur Backend devait valider que son endpoint fonctionnait correctement, respectait la sécurité (vérification du jeton JWT), et retournait les données dans le format JSON attendu. Ce processus garantissait une interface de service stable avant la phase d'intégration avec Angular.

L'Intégration et le Contrôle de Version

Le travail collaboratif a été géré via la plateforme GitHub pour le contrôle de version. Utilisant des branches dédiées pour le développement des fonctionnalités.

- Branches de Fonctionnalités : Chaque fonctionnalité majeure ou User Story était développée sur une branche isolée (feature/nom-de-la-tâche).
- Revue de Code (Pull Requests) : À la fin de chaque tâche, une Pull Request (PR) était soumise. C'était l'occasion pour les encadrants et les autres membres de l'équipe de revoir le code, d'assurer la conformité aux normes de codage et de valider la logique métier avant la fusion dans la branche principale (develop ou main).

Ce processus d'intégration via Git, bien que simple, a permis de maintenir la qualité du code au sein de l'architecture monolithique et d'éviter les régressions majeures avant la livraison finale.

3.2. Conception et Implémentation du Socle Technique Monolithique

Cette section détaille les choix architecturaux et la mise en œuvre des technologies clés qui forment le socle de l'application. L'option pour une architecture monolithique a été retenue pour sa simplicité de déploiement initial et sa rapidité de développement, adaptée à la phase de prototypage et de MVP (Minimum Viable Product).

3.2.1. Mise en Place du Backend avec Spring Boot (Java 21)

Le backend est le cœur de la logique métier. Le choix de Spring Boot (Java 21) repose sur sa robustesse éprouvée dans les applications d'entreprise et sa capacité à gérer les charges transactionnelles et les intégrations complexes comme l'IA.

Configuration et Dépendances Clés

La configuration initiale du projet a nécessité l'intégration de dépendances spécifiques via Maven:

- Spring Data JPA : Essentiel pour la gestion de la persistance des données et le mapping objet-relationnel avec PostgreSQL, basé sur le Diagramme de Classes.
- Spring Web : Pour la création des contrôleurs RESTful gérant les requêtes HTTP entre le Frontend Angular et le Backend.
- Spring Security : Le pilier de la gestion de l'authentification et de l'autorisation.

- PostgreSQL Driver : Le connecteur assurant l'accès à la base de données.

La Couche de Persistance des Données (PostgreSQL)

L'utilisation de PostgreSQL a été privilégiée pour ses fonctionnalités avancées et sa fiabilité dans la gestion de l'intégrité des données, cruciale pour une plateforme de services. Chaque entité définie dans le modèle UML (Expert, Client, Demande, Message, etc.) a été implémentée comme une classe @Entity dans Spring Boot. Par exemple, l'entité Expert contenait des champs pour les données de profil, les compétences, et les métadonnées nécessaires à l'IA. Les Repositories JPA ont permis d'effectuer les opérations CRUD (Create, Read, Update, Delete) avec une abstraction minimale du SQL.

3.2.2. Sécurité et Gestion des Accès : Implémentation de JWT

La sécurité est primordiale pour garantir la confidentialité des communications et la gestion des rôles utilisateur. J'ai configuré Spring Security pour utiliser le mécanisme de Jetons Web JSON (JWT), une solution sans état adaptée aux applications Web modernes.

Le Flux d'Authentification JWT

Le processus d'authentification suit plusieurs étapes rigoureuses, que j'ai implémentées durant la Semaine 2 :

1. Requête de Connexion : L'utilisateur (Client ou Expert ou Admin) envoie ses identifiants au Endpoint de connexion.
2. Génération du Token : Après validation des identifiants dans la base de données, Spring Security génère un JWT signé. Ce jeton encapsule des informations de base sur l'utilisateur (le payload), notamment son ID et son Rôle (Client ou Expert ou Admin).
3. Stockage Côté Client : Le JWT est renvoyé au Frontend Angular, qui le stocke sessionStorage.
4. Accès Sécurisé : Pour chaque requête subséquente (par exemple, pour afficher la liste des experts ou envoyer un message), le Frontend inclut le JWT dans l'entête HTTP (Authorization: Bearer <token>).
5. Validation du Token : Le filtre de Spring Security intercepte la requête, valide la signature du token, et autorise ou rejette l'accès à la ressource en fonction des rôles définis dans le JWT.

Ce mécanisme garantit une sécurité robuste des endpoints REST, essentielle pour séparer les droits d'accès entre les Administrateurs, les Experts et les Clients.

3.2.3. Développement Frontend Modulaire avec Angular 16

Le rôle du Frontend était de garantir une Expérience Utilisateur (UX) fluide et intuitive, en intégrant une grande quantité de données et de fonctionnalités temps réel.

Architecture par Composants et Routage

Angular 16, choisi pour son architecture par composants et sa modularité, a permis de structurer l'application en modules isolés (ex: AuthModule, ProfileModule, ChatModule).

- J'ai travaillé à la mise en place du système de routage pour gérer les vues par rôle utilisateur (Admin Dashboard, Espace Client, Profil Expert), assurant que seul le contenu autorisé soit affiché après authentification (Semaine 4).

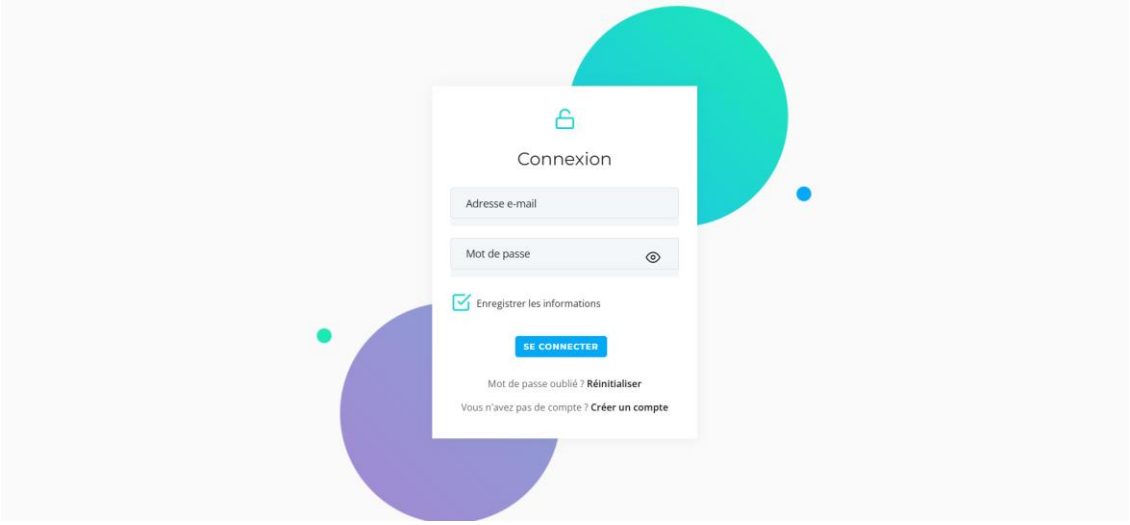

Figure 7 – Interface de connexion avec gestion des rôles utilisateurs (Admin, Expert, Client)

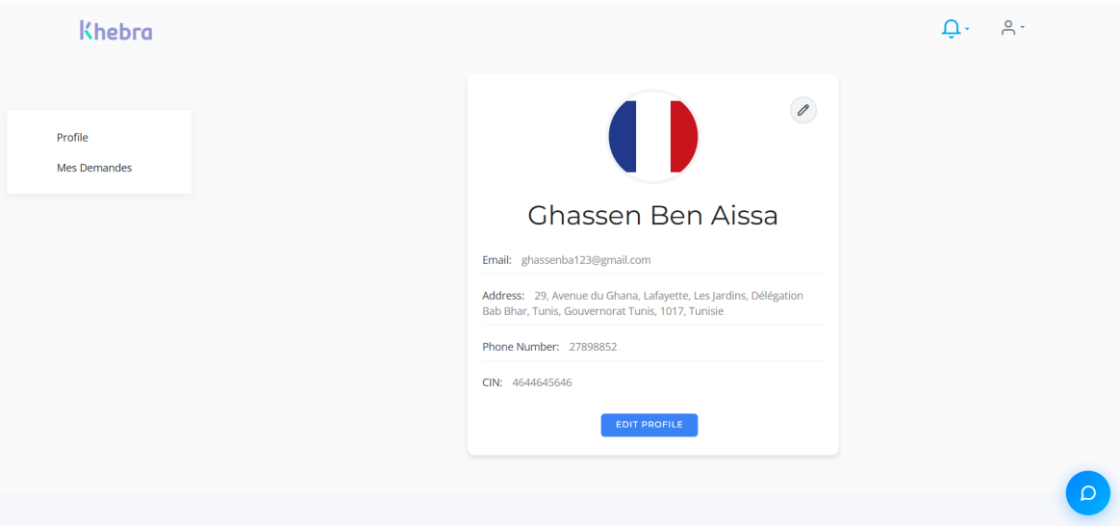

Figure 8 – Interface de navigation dynamique selon le rôle de l'utilisateur

Défis de l'UI/UX pour les Profils et les Évaluations

L'intégration des templates et la création des interfaces (Semaine 3) ont nécessité des ajustements constants pour garantir l'aspect Responsive sur tous les appareils. J'ai particulièrement veillé à la conception des interfaces d'évaluation (système d'étoiles) et des tableaux de bord pour les experts, en veillant à la clarté et à la lisibilité des métriques de performance.

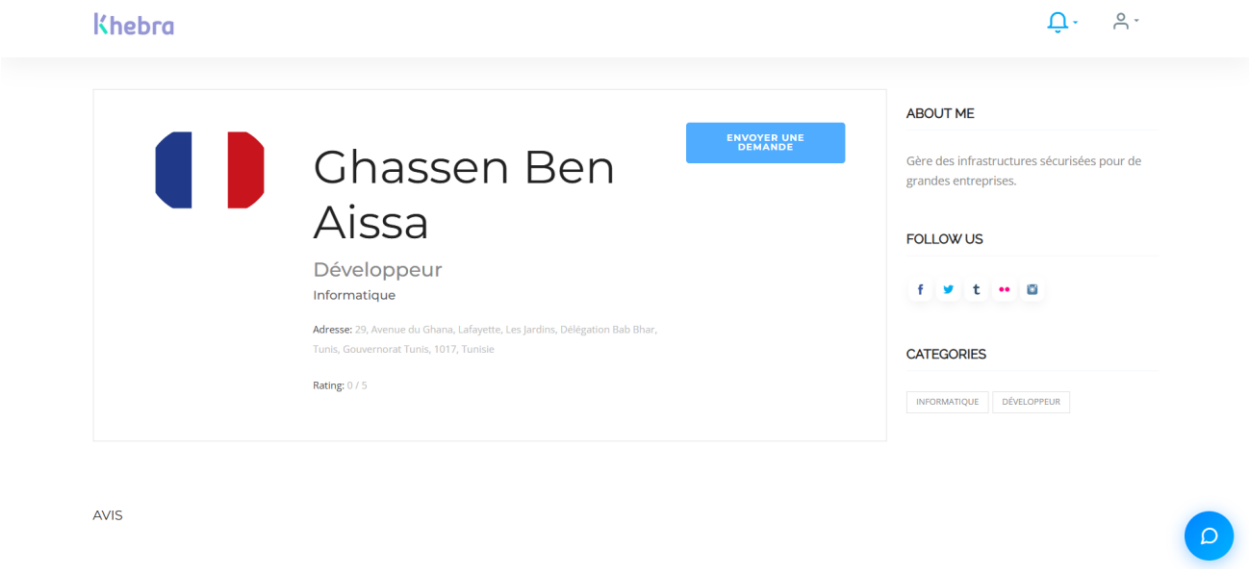

Figure 9 – Détails du profil expert avec biographie, compétences et domaine d'expertise

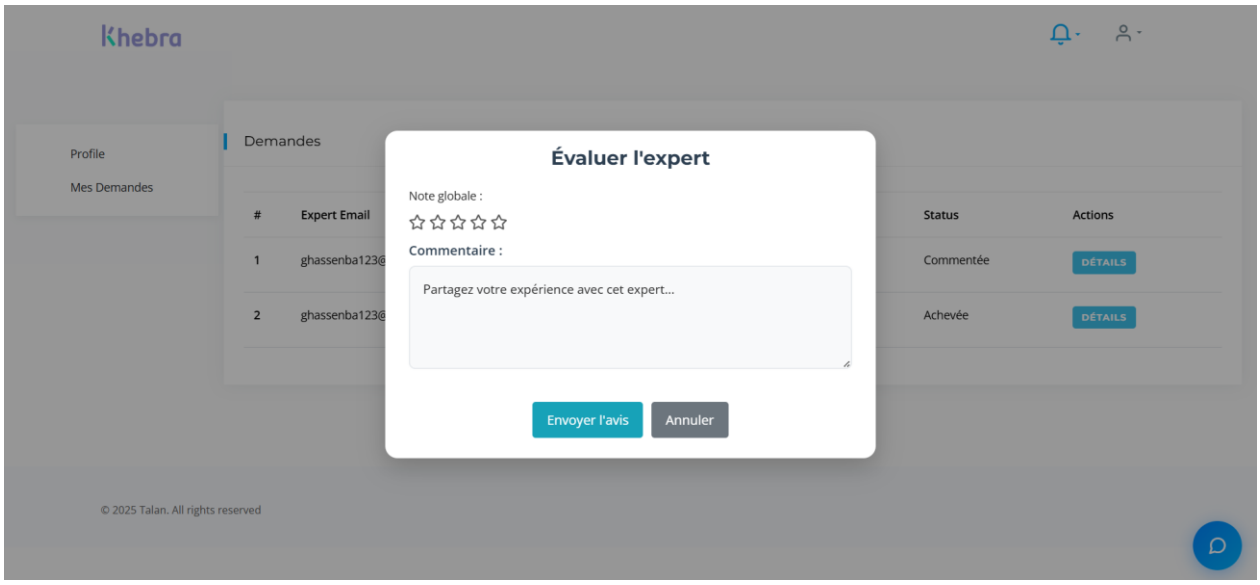

Figure 10 – Système d'étoiles pour l'évaluation d'un expert après une mission

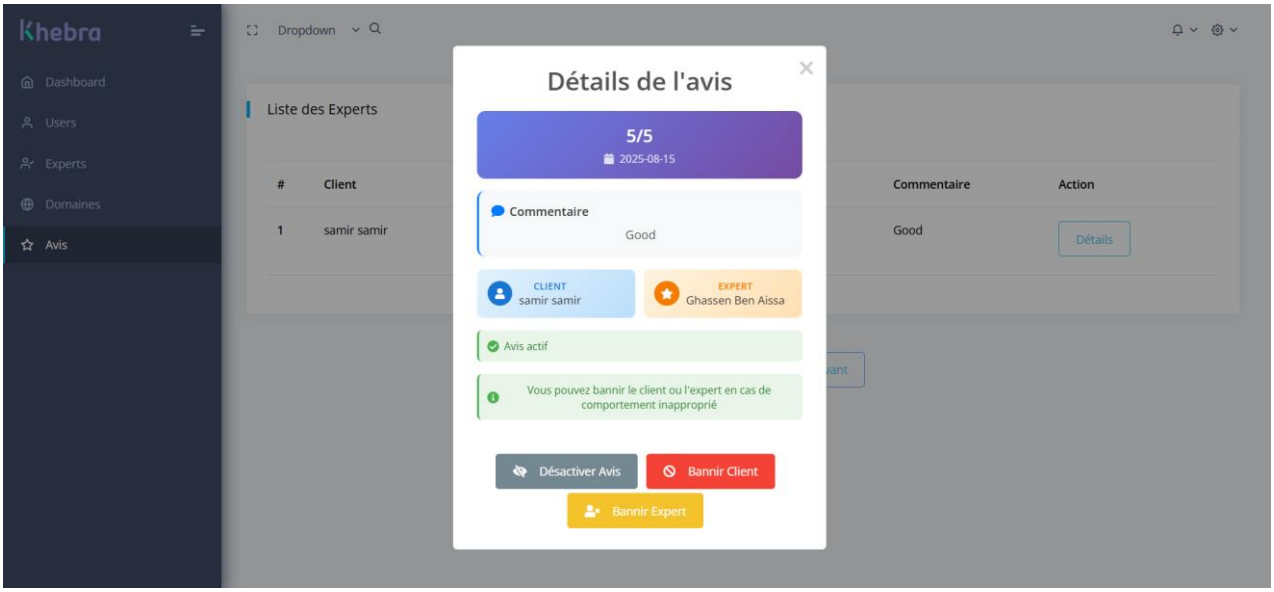

Figure 11 – Dashboard expert affichant les statistiques et les demandes reçues

3.3. Développement des Fonctionnalités Métier et Intégration Avancée

Ce chapitre détaille l'implémentation des fonctionnalités les plus critiques, notamment celles nécessitant une gestion du temps réel et l'intégration de services externes (IA).

3.3.1. Le Système de Communication en Temps Réel (WebSockets)

Le module de messagerie interne était une exigence clé. Un système de communication classique basé sur le polling HTTP aurait généré une surcharge de requêtes. La solution a été l'implémentation du protocole WebSocket.

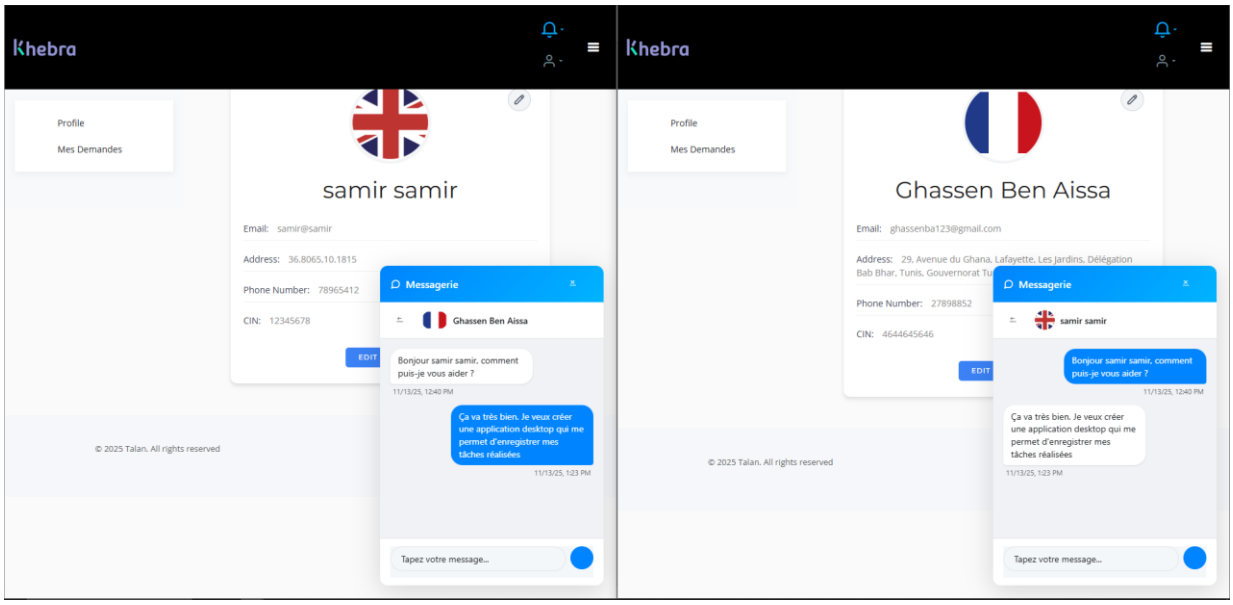

Figure 12 – Espace de discussion entre client et expert (messagerie en temps réel)

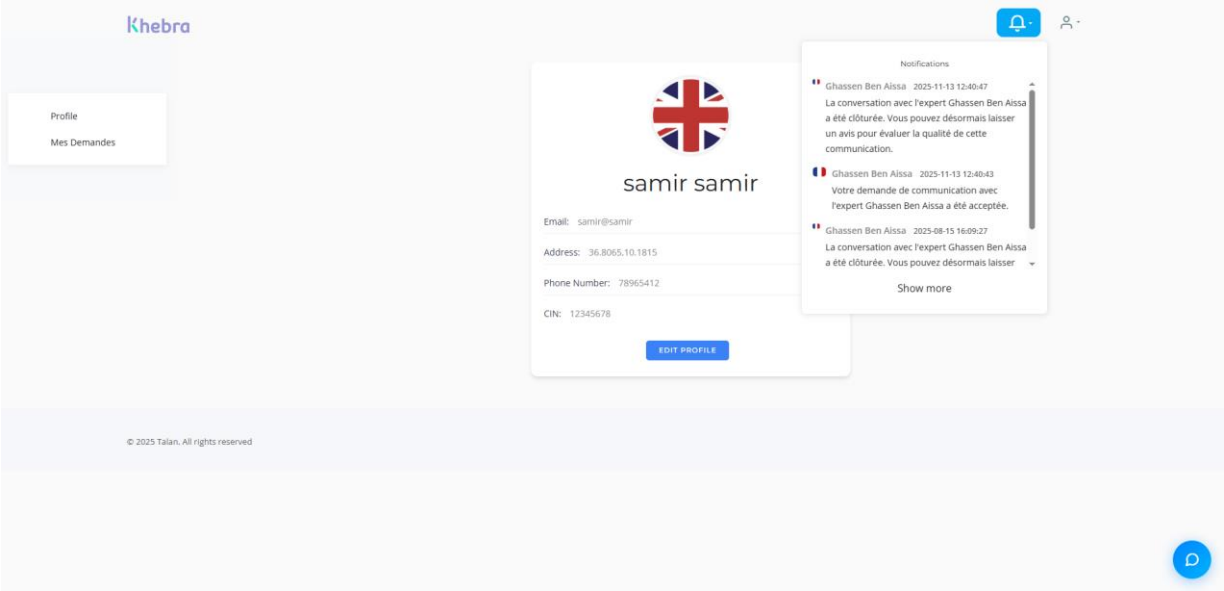

Figure 13 – Notification reçue lors d'une nouvelle demande de communication ou d'un message entrant

Mise en Œuvre de WebSocket avec Spring

- Côté Backend (Spring Boot) : J'ai utilisé l'implémentation Spring de STOMP (Simple Text Oriented Messaging Protocol) sur WebSocket. Le serveur est configuré pour écouter un endpoint WebSocket et gérer les abonnements des clients aux canaux de communication (ex: un canal spécifique à chaque conversation Client-Expert).
- Côté Frontend (Angular) : Le module de messagerie se connecte au serveur WebSocket, permettant l'envoi et la réception instantanés des messages et des notifications de typing.

Ce choix a non seulement permis la création de l'interface de chat mais a également servi de socle pour le système de notifications instantanées des nouvelles demandes de communication.

3.3.2. Architecture d'Intégration de l'Intelligence Artificielle (IA)

L'intégration des modèles d'IA (Sentence Transformer et KNN) dans l'application Java/Spring Boot a été le défi technique majeur du stage, nécessitant une architecture hybride.

Le Modèle de Matching Sémantique

1. Vectorisation Sémantique (Sentence Transformer) : Au lieu d'utiliser des mots-clés, la recherche intelligente repose sur l'Analyse du Langage Naturel (NLP). Sentence Transformer est utilisé pour transformer le texte du besoin client (ex: "J'ai besoin d'un spécialiste en architecture microservices pour le secteur bancaire") en un vecteur dense de dimensions. La même opération est effectuée pour les compétences de chaque expert.
2. Tri par Voisinage (KNN) : L'algorithme K-Nearest Neighbors (KNN) est ensuite appliqué. Il calcule la distance cosinus entre le vecteur du besoin client et tous les vecteurs des experts stockés. Plus la distance est faible, plus le matching est pertinent. KNN permet ainsi de classer les experts du plus pertinent au moins pertinent.

Interfaçage entre Java (Spring Boot) et Python (FastAPI)

Puisque les modèles d'IA sont développés en Python, une couche intermédiaire de service a été indispensable pour la communication :

- Le Service FastAPI (Python) : J'ai créé un service REST API dédié en Python utilisant le framework FastAPI. Ce service est léger, rapide, et a pour unique responsabilité de :
    - Recevoir la description du besoin client (texte) via une requête HTTP POST.
    - Charger et exécuter le modèle Sentence Transformer.
    - Exécuter le modèle KNN sur la base de données des vecteurs d'experts.
    - Retourner au Spring Boot une liste d'Identifiants (IDs) des experts classés.

- Le Flux d'Appel (Spring Boot) : Lorsque le Frontend demande une recherche intelligente, l'API Spring Boot exécute un appel HTTP synchrone au service FastAPI. Une fois la liste des IDs reçue, Spring Boot interroge PostgreSQL pour récupérer les données complètes des profils, et les renvoie au Frontend.

Figure 14– Formulaire de recherche textuelle par le client (rédaction du besoin)

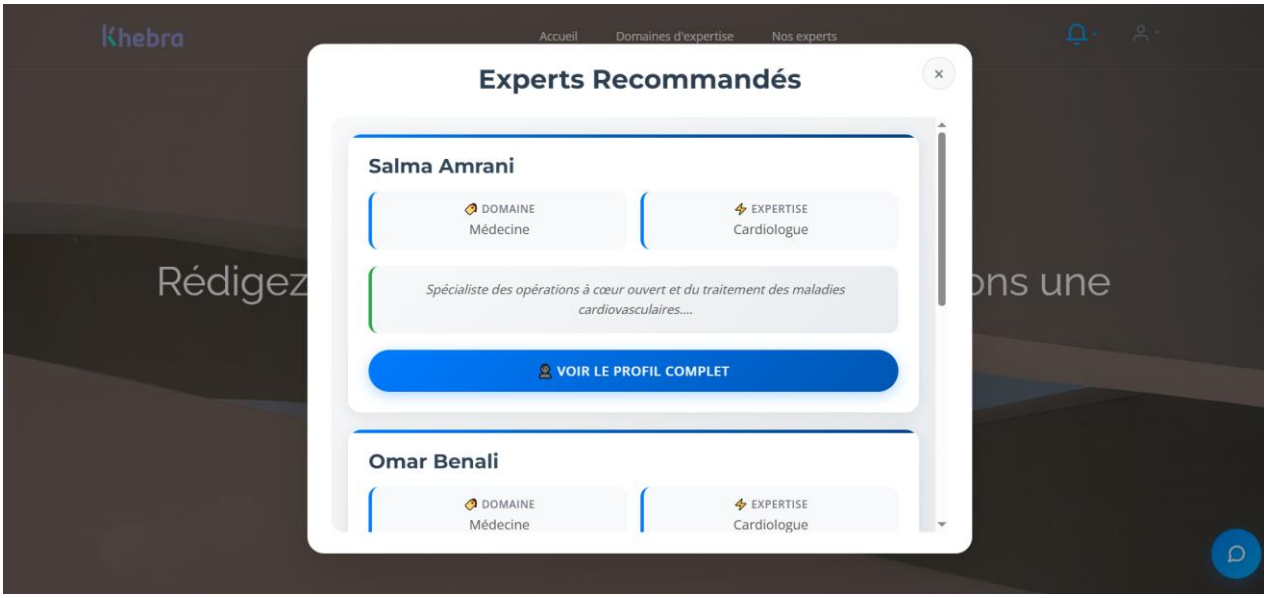

Figure 15 – Liste des experts recommandés avec score de similarité

L'Automatisation de l'Entraînement (Scheduler)

L'efficacité du matching dépend de la fraîcheur des données. J'ai donc mis en place (Semaine 8) un Scheduler dans Spring Boot. Cette tâche planifiée:

1. S'exécute automatiquement toutes les 12 heures.
2. Déclenche un processus de mise à jour des vecteurs d'experts, notamment lorsque de nouveaux experts s'inscrivent ou mettent à jour leurs compétences.

3. Cela garantit que les données utilisées par l'algorithme KNN sont toujours à jour sans nécessiter d'intervention manuelle, assurant la fiabilité et l'évolutivité du système.

3.4. Bilan des Résultats Obtenus

Les réalisations du stage ont permis de livrer un prototype fonctionnel et techniquement complexe.

- Objectif atteint : La plateforme répond aux exigences de base de la mise en relation, avec des fonctionnalités avancées comme la messagerie temps réel et surtout la recherche basée sur l'IA.
- Validation de la Problématique : L'intégration réussie de la couche Python/IA dans l'architecture Java a prouvé qu'il est possible de garantir la robustesse du Backend (Spring Boot) tout en intégrant des technologies tierces pour les besoins de scalabilité et de performance de l'IA.

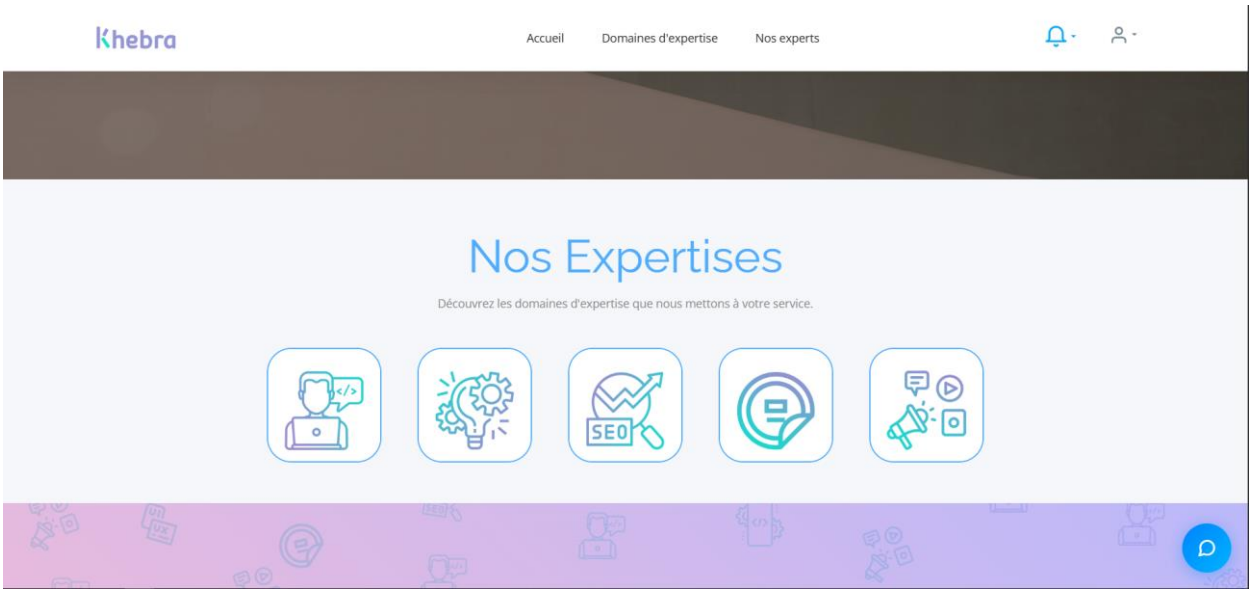

Figure 16 – Interface affichant l'ensemble des domaines disponibles sur la plateforme

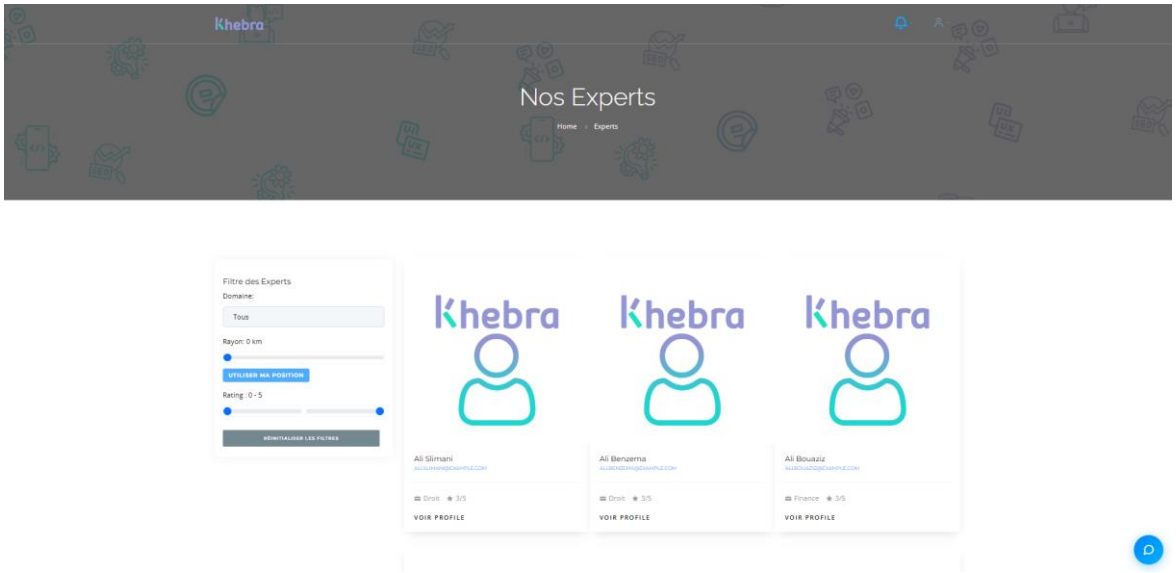

Figure 17 – Présentation des experts inscrits, avec leurs compétences principales

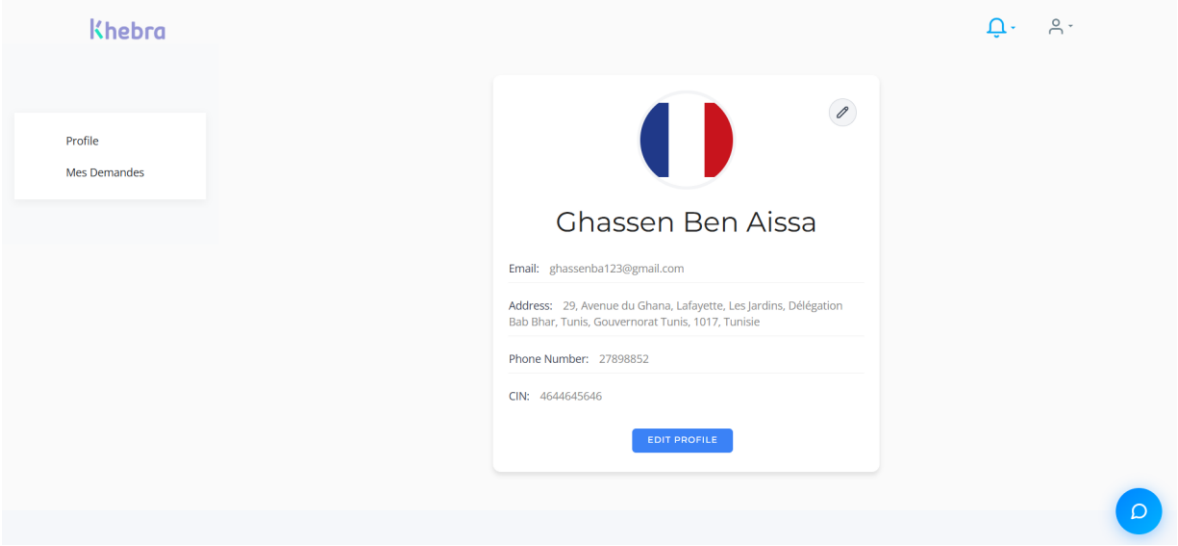

Figure 18 – Détails du profil expert

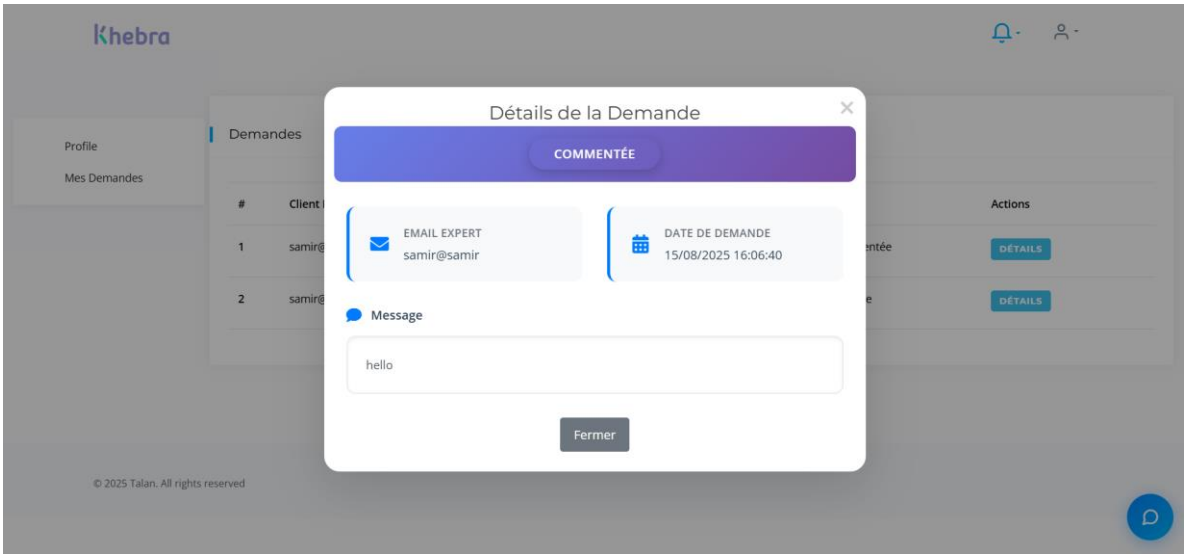

Figure 19 – Liste des demandes (côté expert)

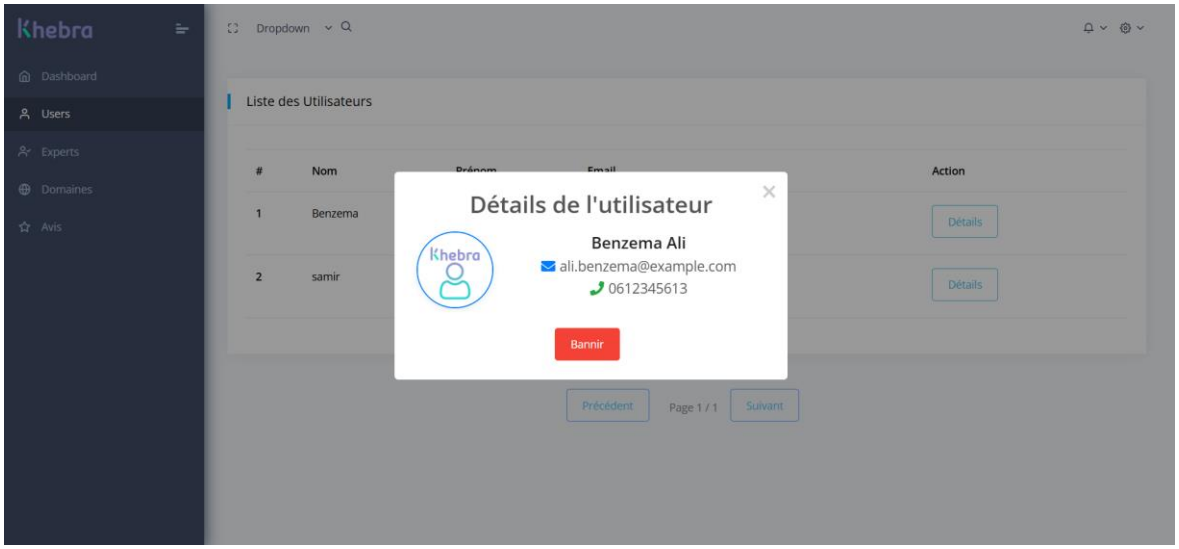

Figure 20 – Détails du client (partie administrateur)

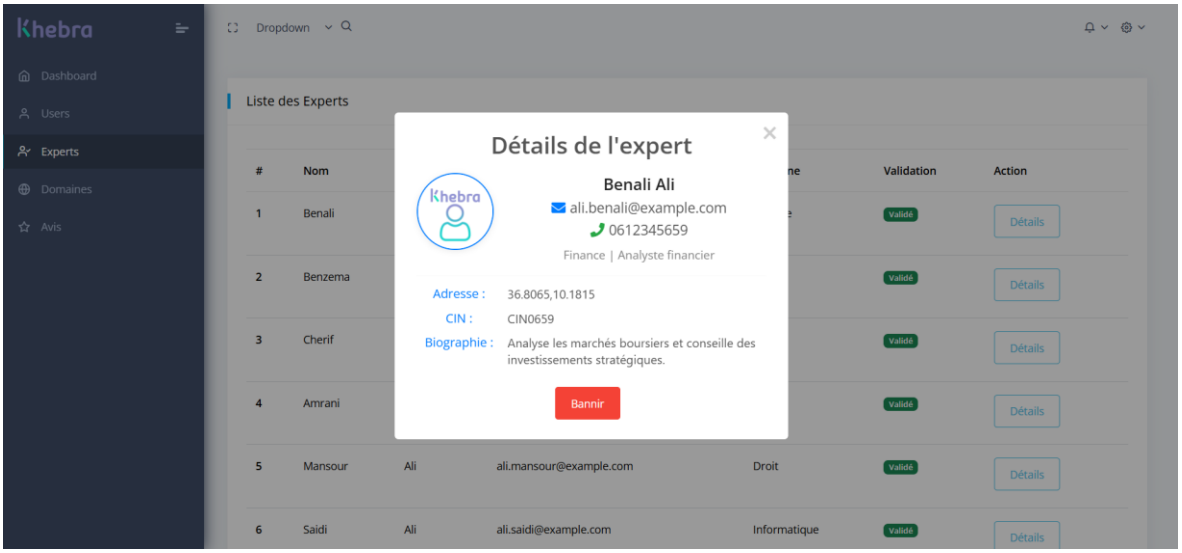

Figure 21 – Détails de l'expert (partie administrateur)

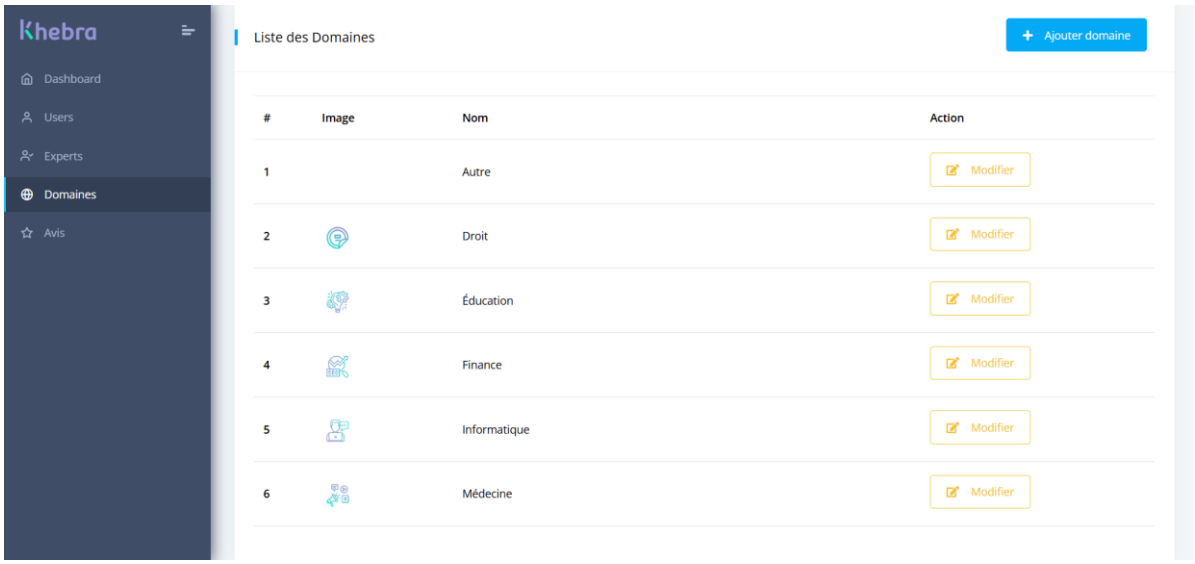

Figure 22 – Liste des domaines (partie administrateur)

Conclusion

J'ai effectué mon stage de 8 semaines au sein de l'entreprise Talan Tunisie, sous la supervision de Mr. Houssem Jmal. Cette expérience m'a permis d'appliquer les connaissances théoriques acquises au cours de ma formation tout en me confrontant aux défis réels du monde professionnel. Je garde un excellent souvenir de ce stage, qui représente une expérience précieuse et motivante pour mon avenir. Il m'a offert une préparation solide pour ma future carrière et a enrichi mon parcours professionnel.

Signature du maître de stage	Cachet de l'entreprise
	TALAN TUNISIE CONSULTING Rue de l'Energie Solaire, Impasse 1 2035 Chargula 1 Tél: 70.015.010 - Fax: 70.015.007

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : 1-2 rue André Ampère - 2083 - Pôle Technologique - El Ghazala - Tél +216 70 250 000 - Fax +216 70 685454